Assignment 4

🤖 Artificial Intelligence for Robotics

Re-use and update the module Jupyter notebook. Each cell prints metrics and draws figures. Run it top to bottom before you start. Every method in there is greedy, and every greedy method gets stuck — on a peak, in a force dead-spot, or wandering a maze it cannot see. You will push each demo until it fails, then add the one non-greedy ingredient that rescues it (a restart, a swirl, a coordinated behavior, a learned cost-to-go) and measure what the fix actually bought. Seed your Randg (RNG) and average stochastic results; every plot gets a title, axes, and a legend.

1. [Annealing, 20 pts] Plain hill climbing finds the global optimum on the local search landscape only about a quarter of the time, because its outcome is decided by the starting basin.

   (a) Add a random-restart wrapper. From your measured per-run success rate `p`, predict the restart budget needed for 95% success; verify it empirically and plot success vs. budget.

   (b) Give random restarts and simulated annealing the same total number of objective-function evaluations and compare success rate and best value. Which wins on this landscape, and what feature of a landscape would flip the answer?

2. [Potential fields, 20 pts] A potential field stalls wherever attraction and repulsion cancel.

   (a) Design an obstacle layout (e.g., a concave "U") that traps the notebook's `navigate()`, and show the stall with a quiver plot and a run.

   (b) Compare two escapes — Murphy's tangential primitive vs. a random kick injected when the resultant force goes slack (the field analogue of annealing) — on success rate and path length over ≥30 runs.

   (c) In a few sentences: why can no purely reactive field guarantee escape from a U-trap, and what minimal added capability (memory, sensing, or planning) fixes it?

3. [Coordinating behaviours, 30 pts] Instead of summing one field, coordinate several simple behaviors.

   (a) Implement GO-TO-GOAL and FOLLOW-WALL behaviors and a finite-state automaton that switches between them; draw the state-transition diagram and color-code the active behavior along a successful run through your U-trap.

   (b) Recast the same controller in subsumption terms (layers, suppression, fixed priority) and contrast competing (FSA / subsumption) with cooperating (potential-field summation) coordination — Murphy Ch. 8. Which was easier to design, and which was more robust on your traps?

   (c) Add a script (a fixed recovery maneuver) triggered after the robot is stuck for N steps, and report whether it helps on a harder layout.

4. [Searching an unknown map, 30 pts] A* needs the map up front; LRTA* earns it one sensed cell at a time.

(a) Generate solvable mazes at a few sizes (e.g., 12^2, 20^2, 32^2); plot LRTA*'s competitive ratio (distance travelled and the A* optimum) vs. size, averaged over several mazes, and describe the trend.

(b) Pit LRTA* against a backtracking online DFS on the same mazes. Show a revisit heatmap for one maze and explain how rising H-values push the agent out of dead ends.

(c) State the conditions under which LRTA* is guaranteed to reach the goal, and one grid property that can force any online agent into an arbitrarily large competitive ratio.